

**Technical Design Document**  
**Life Science Intelligence Center**

## SOURCE → TARGET MAPPING (ACTUAL)

### 1. Patient Mapping

| Source     | Silver        | Gold                   |
|------------|---------------|------------------------|
| patient_id | patient_id    | dim_patient.patient_id |
| gender     | UPPER(gender) | dim_patient.gender     |
| age        | INT           | dim_patient.age        |
| diagnosis  | INITCAP       | dim_diagnosis          |
| site_id    | FK validated  | used in facts          |

#### Derived

age\_group =

CASE

WHEN age < 30 THEN 'Young'

WHEN age < 60 THEN 'Adult'

ELSE 'Senior'

END

### 2. Visit Mapping

| Source     | Silver   | Gold       |
|------------|----------|------------|
| visit_id   | visit_id | fact_visit |
| patient_id | FK       | fact_visit |
| visit_date | TO_DATE  | dim_date   |
| visit_type | INITCAP  | fact_visit |
| site_id    | FK       | fact_visit |

### 3. Enrollment Mapping

| Source          | Silver  | Gold            |
|-----------------|---------|-----------------|
| enrollment_id   | PK      | fact_enrollment |
| patient_id      | FK      | fact_enrollment |
| trial_id        | FK      | fact_enrollment |
| site_id         | FK      | fact_enrollment |
| enrollment_date | DATE    | dim_date        |
| status          | INITCAP | fact_enrollment |

#### Derived

enrolled\_flag = 1 if status in ('Enrolled','Completed')

### 4. Lab Result Mapping

| Source        | Silver  | Gold         |
|---------------|---------|--------------|
| lab_result_id | PK      | fact_lab     |
| test_name     | UPPER   | dim_lab_test |
| result_value  | DECIMAL | fact_lab     |
| result_date   | DATE    | dim_date     |

#### Derived

pre\_visit\_flag = result\_date < visit\_date

### 5. Biomarker Mapping

| Source           | Silver  | Gold           |
|------------------|---------|----------------|
| biomarker_id     | PK      | fact_biomarker |
| marker_name      | UPPER   | dim_biomarker  |
| expression_level | DECIMAL | fact_biomarker |

### 6. Adverse Event Mapping

| Source     | Silver  | Gold    |
|------------|---------|---------|
| ae_id      | PK      | fact_ae |
| event_type | INITCAP | dim_ae  |

| Source   | Silver  | Gold    |
|----------|---------|---------|
| severity | INITCAP | fact_ae |

### Derived

severity\_score =

Mild=1, Moderate=2, Severe=3

## 7. Claims Mapping

| Source       | Silver  | Gold       |
|--------------|---------|------------|
| claim_id     | PK      | fact_claim |
| claim_amount | DECIMAL | fact_claim |
| claim_status | INITCAP | fact_claim |
| visit_id     | FK      | join visit |

### PYSPARK CODE

#### CONFIG

CATALOG = "lsic\_dev"

BRONZE = f"{CATALOG}.bronze"

SILVER = f"{CATALOG}.silver"

GOLD = f"{CATALOG}.gold"

LANDING = "s3://lsic-dev/landing"

CHECKPOINT = "s3://lsic-dev/checkpoints"

#### ◇ 1. AUTO LOADER → BRONZE

```
from pyspark.sql.functions import *
```

```
from pyspark.sql.types import *
```

```
def ingest(entity):
```

```
    df = (
```

```
        spark.readStream
```

```

        .format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load(f"{LANDING}/{entity}")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )

    return (
        df.writeStream
        .format("delta")
        .option("checkpointLocation", f"{CHECKPOINT}/{entity}")
        .trigger(availableNow=True)
        .toTable(f"{BRONZE}.{entity}_bz")
    )

# Run ingestion
for t in
["ehr_patient", "ehr_visit", "trial", "site", "enrollment", "lab_result", "biomarker", "adverse_event", "claims
"]:
    ingest(t)

```

## 2. SILVER TRANSFORMATIONS

### Patient

```

df = spark.table("lsic_dev.bronze.ehr_patient_bz")

patient = df.select(
    trim(col("patient_id")).alias("patient_id"),
    upper(col("gender")).alias("gender"),
    col("age").cast("int"),
    initcap(col("diagnosis")).alias("diagnosis"),
    trim(col("site_id")).alias("site_id")
)

```

```
patient = patient.withColumn(  
    "age_group",  
    when(col("age") < 30, "Young")  
    .when(col("age") < 60, "Adult")  
    .otherwise("Senior")  
)
```

```
patient.write.mode("overwrite").saveAsTable("lsic_dev.silver.patient_slv")
```

### **Visit**

```
visit = spark.table("lsic_dev.bronze.ehr_visit_bz") \  
.withColumn("visit_date", to_date("visit_date", "dd-MM-yyyy")) \  
.withColumn("visit_type", initcap("visit_type"))
```

```
visit.write.mode("overwrite").saveAsTable("lsic_dev.silver.visit_slv")
```

### **Lab Result**

```
lab = spark.table("lsic_dev.bronze.lab_result_bz") \  
.withColumn("result_date", to_date("result_date", "dd-MM-yyyy"))
```

```
visit = spark.table("lsic_dev.silver.visit_slv")
```

```
lab = lab.join(visit.select("visit_id", "visit_date"), "visit_id") \  
.withColumn("pre_visit_flag", col("result_date") < col("visit_date"))
```

```
lab.write.mode("overwrite").saveAsTable("lsic_dev.silver.lab_slv")
```

### **Biomarker**

```
bio = spark.table("lsic_dev.bronze.biomarker_bz") \  

```

```
.withColumn("marker_name", upper("marker_name"))
```

```
bio.write.mode("overwrite").saveAsTable("lsic_dev.silver.biomarker_slv")
```

### **Adverse Event**

```
ae = spark.table("lsic_dev.bronze.adverse_event_bz") \  
.withColumn("severity_score",  
  when(col("severity")=="Mild",1)  
  .when(col("severity")=="Moderate",2)  
  .otherwise(3)  
)
```

```
ae.write.mode("overwrite").saveAsTable("lsic_dev.silver.ae_slv")
```

## **GOLD LAYER (SQL)**

### **Dimension: Patient**

```
CREATE OR REPLACE TABLE lsic_dev.gold.dim_patient AS  
SELECT  
  patient_id,  
  gender,  
  age,  
  age_group  
FROM lsic_dev.silver.patient_slv;
```

### **Fact: Enrollment**

```
CREATE OR REPLACE TABLE lsic_dev.gold.fact_enrollment AS  
SELECT  
  e.enrollment_id,  
  p.patient_id,  
  t.trial_id,  
  s.site_id,
```

```

e.enrollment_date,
CASE WHEN status IN ('Enrolled','Completed') THEN 1 ELSE 0 END AS enrolled_flag
FROM lsic_dev.silver.enrollment_slv e
JOIN lsic_dev.silver.patient_slv p USING(patient_id)
JOIN lsic_dev.silver.trial_slv t USING(trial_id)
JOIN lsic_dev.silver.site_slv s USING(site_id);

```

### **Fact: Lab**

```

CREATE OR REPLACE TABLE lsic_dev.gold.fact_lab AS
SELECT
l.lab_result_id,
l.patient_id,
l.test_name,
l.result_value,
l.result_date,
l.pre_visit_flag
FROM lsic_dev.silver.lab_slv l;

```

### **Fact: Biomarker**

```

CREATE OR REPLACE TABLE lsic_dev.gold.fact_biomarker AS
SELECT
biomarker_id,
patient_id,
trial_id,
marker_name,
expression_level
FROM lsic_dev.silver.biomarker_slv;

```

## **DATA QUALITY RULES (IMPLEMENTABLE)**

```
-- Lab anomaly check
```

```
SELECT COUNT(*)  
FROM silver.lab_slv  
WHERE result_date < visit_date;  
-- FK validation  
SELECT *  
FROM silver.lab_slv l  
LEFT JOIN silver.visit_slv v  
ON l.visit_id = v.visit_id  
WHERE v.visit_id IS NULL;
```